

8PRO128 - Programmation orientée objet

Laboratoire #4

Templates, héritage multiple, librairie standard

Automne 2022

Professeure : Sara Séguin

Instructions générales

- Devoir à remettre au plus tard le **8 décembre à 13 heures**.
- Travail à réaliser en équipe de 2 étudiants (ou 3).
- Un seul rapport à remettre par équipe.
- Ce travail compte pour 10% de la note finale.

Énoncé du laboratoire

Le laboratoire comporte plusieurs parties : templates, héritage multiple et librairie standard.

Partie 1 : Templates

Vous devez reprendre l'énoncé du laboratoire 2 et créer une classe générique `Matrice`, c'est-à-dire que n'importe quel type de données peuvent composer la matrice. Pour ce laboratoire précis, les types pouvant être donnés à la classe sont `double`, `int` et `float`. Toutes les fonctionnalités du laboratoire 2 doivent être disponibles dans la classe générique. Pour tester votre matrice dans le main, vous devez montrer que vous avez une seule classe mais que tous les types demandés fonctionnent.

Partie 2 : Héritage multiple

Votre entreprise a obtenu un contrat pour afficher des éléments liés à un jeu sur le web. À cause des clauses de confidentialité, vous ne savez pas exactement ce que vous affichez, alors vous devez définir un code générique permettant d'afficher des rectangles, des points et des cercles.

Vous devez d'abord définir une **classe `Print`** qui est une interface. Elle doit avoir une fonction virtuelle pure `affiche()`. Cette fonction doit être redéfinie dans les classes dérivées pour afficher les caractéristiques de l'objet qui appelle la méthode. Les caractéristiques sont données par un attribut de type booléen qui indique si l'objet est rempli (si `=1`, seul le contour et si `=0`, objet rempli) et par un entier `epaisseur` qui spécifie l'épaisseur du contour.

La **classe `Data`** a 2 attributs privés entiers (`xx` et `yy`) qui donnent respectivement la position `x` et `y` du `Data` dans l'écran. Les méthodes suivantes sont disponibles : constructeur par

défaut, méthode affiche qui permet d'afficher les informations géométriques du Data (aire, périmètre...), une méthode virtuelle pure calculerAire et une méthode virtuelle pure CalculerPerimetre. De plus, les méthodes d'accès et de modification pour tous les attributs doivent être disponibles.

La **classe Dataset** possède un attribut qui est un vecteur de pointeurs de **Data**. L'attribut est vecData, un vecteur de pointeur de Data. Les méthodes suivantes sont disponibles : ajouterData, printData qui affiche la liste des datas stockées dans le vecteur, constructeur par défaut et destructeur.

La **classe Rectangle** dérive de la classe **Print** et de la classe **Data**. Ses attributs sont 4 sommets de type Point qui spécifient les différents sommets du rectangle. Les méthodes suivantes sont disponibles : constructeur par défaut, constructeur par paramètres (4 sommets en paramètres), getLargeur, getLongueur, calculerAire, calculerPerimetre, affiche (longueur, largeur, aire, périmètre, son importance et son attribut booléen.)

La classe **Point** a 2 attributs double xx et yy qui donne les coordonnées de l'objet. Les méthodes disponibles sont : constructeur par paramètres (coordonnées données en paramètres), constructeur par défaut (xx=0 et yy=0), distance qui donne la distance entre 2 points, méthodes de modification et d'accès pour les attributs et affiche pour afficher les coordonnées.

La classe **Cercle** dérive de la classe **Print** et de la classe **Data**. Ses attributs sont centre (pointeur sur un objet de type Point) et rayon. Les deux sont de type double. Les méthodes suivantes sont disponibles : constructeur par paramètres, calculerAire, calculerPerimetre, affiche (rayon, centre, aire, périmètre, son importance et son attribut booléen).

Pour tester votre main.cpp, vous devez tester la classe Dataset. Ainsi, vous devez créer des objets de type rectangle, point et cercle et les ajouter à Dataset. Détaillez votre méthodologie dans le rapport. Pour vous aider, dessinez le diagramme de classes.

Partie 3 : Librairie standard

Un fichier contenant 1000 valeurs entières vous est remis (« input.txt»). Vous devez lire les 1000 valeurs, puis les transformer en valeur hexadécimale. Afin d'y parvenir, vous devez créer une ou des classes avec les bons attributs et les méthodes nécessaires. N'oubliez pas de créer les méthodes set et get pour tous les attributs. Lorsque les 1000 adresses auront été traduites, vous devez les écrire dans un fichier « solution.txt » et les afficher à l'écran.

Afin d'y parvenir, vous devez utiliser les conteneurs, itérateurs, foncteurs et algorithmes de la librairie STL. Vous devez ainsi produire un code avec le moins de lignes possibles en

utilisant le plus de fonctionnalités de la STL. Vous devez détailler dans le rapport les différents éléments que vous avez utilisé et justifier vos choix.

Travail à effectuer

1. Vous devez créer un fichier d'en-tête (.h) et un fichier source (.cpp) pour chaque classe et qui répondent à tous les éléments mentionnés pour chaque partie.
2. Vous devez vous créer un fichier main.cpp qui permet de tester toutes les méthodes disponibles, pour chaque partie. À vous de proposer une méthodologie qui vérifie tous les éléments du laboratoire!

Considérations

- Utilisation de const lorsque nécessaire.
- Destructeurs et destructeurs virtuels.
- Passages par paramètres/références.
- Utilisation de virtual lorsque nécessaire.

À remettre

Vous devez préparer un rapport de devoir (un seul rapport pour toutes les sections) comportant :

- Page-titre!
- Brève introduction qui explique le but du laboratoire.
- Vous devez expliquer votre démarche pour la réalisation de votre programme dans une section méthodologie. Vous devez aussi détailler votre main.cpp et comment sont validés tous les éléments du laboratoire, POUR CHAQUE PARTIE.
- Une section résultats. Faites des captures d'écran des fonctionnalités demandées.
- Une conclusion.
- Ajouter le code en annexe de votre rapport. Une section pour les .h, une section pour les .cpp et une pour le main.
- Vous devez remettre une copie électronique de votre travail.
- Votre code doit être commenté!